P2452R0
2021 October Library Evolution and Concurrency Networking and Executors Poll Outcomes

Published Proposal, 2022-02-15

Authors:

Bryce Adelstein Lelbach (he/him/his) — Library Evolution Chair (NVIDIA)

Fabio Fracassi — Library Evolution Vice Chair (CODE University of Applied Sciences)

Ben Craig — Library Evolution Vice Chair (NI)

Source:

GitHub

Issue Tracking:

GitHub

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:

WG21

Table of Contents

§ 1. Introduction

In October 2021, the C++ Library Evolution group conducted a series of electronic decision polls on Networking and Executors [P2452R0]. This paper provides the results of those polls and summarizes the results.

In total, 56 people participated in the polls. Some participants opted to not vote on some polls. Thank you to everyone who participated, and to the proposal authors for all their hard work!

§ 2. Poll Outcomes

- SF: Strongly Favor.
- WF: Weakly Favor.
- N: Neutral.
- WA: Weakly Against.
- SA: Strongly Against.

Poll	SF	WF	N	WA	SA	Outcome
Poll 1: The Networking TS/Asio async model (P2444) is a good basis for most asynchronous use cases, including networking, parallelism, and GPUs.	5	10	6	14	18	Weak consensus against. What this doesn't mean: It doesn't mean that the Networing TS async model isn't a good fit for networking. There were many comments to the contrary. What this means: If the authors continue to pursue a design similar to the current Networking TS, they would have an easier time getting consensus by focusing on the networking side of

						things, rather than proposing the design for general asynchrony.
Poll 2: The sender/receiver model (P2300) is a good basis for most asynchronous use cases, including networking, parallelism, and GPUs.	24	16	3	6	3	Consensus in favor. What this doesn't mean: It doesn't mean that S&R is going into C++23 (though it might). It doesn't mean that [P2300R2] as it stands today will be sent forward to Library. It doesn't prevent us from adding new asynchronous models in the future. What this means: Work will continue in Library Evolution on refining [P2300R2], and Library Evolution will keep the various asynchronous use cases in mind while working on [P2300R2].

Poll 3: Stop pursuing the Networking TS/Asio design as the C++ Standard Library's answer for networking.	13	13	8	6	10	No consensus. What this doesn't mean: The Networking TS is not "dead". What this means: The C++ Committee will still work on networking in this general form, but the authors need to do a lot in order to build up consensus to get something like the TS merged into the standard. The bulk of this work should be done in Networking Study Group. Many of the people in favor of stopping work on the TS would like networking to be built on top of Senders and Receivers. Others were opposed to the lack of security through Transport Layer Security (TLS). It is highly unlikely that design changes to the Networking TS can be made fast enough, and consensus gained fast enough, for networking to make C++23.
Poll 4: Networking in the C++ Standard Library should be based on the sender/receiver model (P2300).	17	11	10	4	6	Weak consensus. What this doesn't mean: Work on networking using other models will still be reviewed and considered on its own merits. WG21 doesn't pause work on a concrete paper based off of the wish for another paper. Note that there are a high number of neutrals on this vote. Many of the neutrals (and some of the abstentions) would like to see a paper before picking a side here. What this means: In the short term, this poll result doesn't mean much. We don't have a paper in hand that proposes networking based on the [P2300R2]

						model. For paper authors though, this poll is encouragement to do work in the area of networking based on senders and receivers, or to be prepared with compelling new information on why networking should use a different model.
Poll 5: It is acceptable to ship socket-based networking in the C++ Standard Library that does not support secure sockets (TLS/DTLS).	9	13	5	6	13	No consensus. What this means: A networking library that does not support secure sockets will face significant headwinds getting through the standardization process. What this doesn't mean: This doesn't make a statement on whether insecure networking should be included, the defaults of secure vs. insecure, or how things like ABI should managed.

§ 3. Networking Study Group Guidance

Before bringing networking papers back to Library Evolution, two major areas need to be thoroughly addressed: Security, and the Senders and Receivers async model [P2300R2].

Those voting in favor of Poll 5 argued that the insecure parts are the building blocks for the secure parts, and that ABI is major concern that will plague TLS support. Those voting against Poll 5 argued that secure sockets are needed for communicating with many sites on the internet, and that shipping without secure sockets would be irresponsible. A networking proposal needs to address these concerns before coming to LEWG.

As for networking in combination with Senders and Receivers, the following poll result from the 2021-09-28 telecon may be informative:

Poll	SF	WF	N	WA	SA	Outcome
We believe we need one grand unified model for asynchronous execution in the C++ Standard Library, that covers structured	4	9	5	5	1	No consensus (leaning in favor).

concurrency, event based programming, active patterns, etc.						

The combination of this "grand unified model" poll and Poll 4 heavily encourages the networking study group to produce a paper based on Senders and Receivers. There is room to produce a non-S&R paper, but such a paper would need to provide compelling new information in order to convince the "grand unified model" contingent that S&R can’t get the job done suitably.

4. Selected Poll Comments

4.1. Poll 1: Networking TS Model Is A Good Basis

> I’m voting in favour in this poll because, based on implementation experience, I understand that the Networking TS/Asio model is indeed capable of supporting these use cases.
>
> However, a comment on the poll itself: It appears to presume that we must have a single asynchronous model. In the previous LEWG call dated 2020-09-28, a poll was taken on the question "We must have a single async model for the C++ Standard Library". There was no consensus, and that question was not re-asked before taking this poll.
>
> — Strongly Favor

> Asio/Net.TS is proven in multiple disciplines, many corporations
>
> — Strongly Favor

> This design seems capable of being used by any higher-level concurrency framework, e.g. senders/receivers, so seems like a good basis even if it alone is not sufficient to serve the needs of most users.
>
> — Strongly Favor

> We have proven this in demonstrable code, including the GPU case.
>
> — Strongly Favor

> It’s a mature and well test model and can accomodate more specific models such as sender/receiver.
>
> — Strongly Favor

> Definitely yes for networking, but unsure about GPUs.
>
> — Weakly Favor

> The model seems to corelate with my experience, so, in lack of code examples for all the cases mentioned above, I will weakly assume this is true.
>
> — Weakly Favor

> "Good" is super-subjective... It is an idea that appears to work, so that makes it good in my understanding.
>
> — Weakly Favor

> The TS/Asio model seems well suited to network programming, if a bit cumbersome for GPU programming it doesn’t seem antithetical to it. The interface in [P2300R2] seems more in line with the compute use-cases, and [P2444R0] to async event-driven use-cases. That said, I don’t think the difference is as stark as has been made out.
>
> — Weakly Favor

> The ASIO async model appears to allow for a high degree of control and customization over the way that asynchronous execution occurs.
>
> — Weakly Favor

> I don’t think its great for parallelism/GPUs but its better than what we have now.
>
> — Weakly Favor

> Neither Asio/NetTS nor Sender/Receiver are currently a good basis for bulk parallel compute. Asio/NetTS is a good basis for most other asynchronous use cases, including as an execution agent for Sender/Receiver style workflows.
>
> — Weakly Favor

> I have been on several projects that used Boost.Asio and seen several presentations about using the Asio async method and it seems to fit into the networking use cases well. I cannot comment on other use cases.
>
> — Weakly Favor

> I’m concerned that the Networking TS model as currently reified doesn’t adequately support the kind of zero allocation strategies that S&R’s "operation state" concept enables and therefore can’t vote strongly in favor.
>
> — Weakly Favor

> I like Networking TS for networking. But it’s hard to add own asynchronicity sources.
>
> — Neutral

> I have much more personal experience with prototyping around scheduler/sender/receiver and have not prototyped parallelism using the model in Networking TS. During LEWG/SG1 calls, there have been claims about the suitability or lack of suitability of the networking TS model for various asynchronous use cases. Because of this, we are now beginning an internal evaluation of this model for our use cases but have not yet done this evaluation. I believe the result could go in either direction, and so I’m neutral.
>
> — Neutral

> My vote is neutral as I am not familiar enough with the networking requirements to have an opinion on those, and I have not seen enough of Asio being used for GPU use cases to have an opinion on whether it would satisfy the requirements there.
>
> — Neutral

> It has demonstrated it fits networking. It has not demonstrated it fits GPUs, and I buy Michal's arguments that it does not. I have no first-person experience to evaluate those myself, though.
>
> — Neutral

> Unfortunately didn't have enough time to deeply understand the Networking TS model (that's why I am Neutral) but at first glance it seems to be a good basic for everything that's listed in the poll with some tweaks. Somebody claims that there is no clear error channel and cancellation mechanism that fits necessary use-cases. I tend to believe in those claims but I think this model can be extended to fulfill people expectations.
>
> The very basic asynchronous model (IMO) should have kind of spawn the "task", wait for completion of the "task" (group of tasks), the way to express dependencies between "tasks", the way to dynamically add more "tasks" (create other tasks within some "task" execution), error channel and cancellation. I think that Networking TS looks quite low-level to build everything on top of that and if it's not immediately fulfills the requirements to async model I've described above we may think how it can be extended to support those.
>
> Again, I need to deeper dive into Networking TS to form clearer opinion. I just put some ruminations basing what I've already learned.
>
> — Neutral

> Application of the async model to GPU workloads prevents calling this anything other than neutral.
>
> — Neutral

> Didn't participate in the discussions, so I don't have a worthwhile opinion.
>
> — Did Not Participate

> Despite several presentations on the subject, the [P2444R0] model still seems quite mysterious. Obviously it has found extensive, productive use in practice, but some of its fundamental concepts are complicated and the import of the examples has been obfuscated by (important!) details like coroutine support and by their domain specificity.
>
> — Did Not Participate

> Uninformed and actively abstaining for informational purposes.
>
> — Did Not Participate

> NetTS async model combination of errors and results into a single completion signature is limiting and problematic for cases where an operation fails and I might not be able to construct a result-value, or the result-value might not have a suitable invalid state. The model also seems to either encourage use of shared_ptr for keeping state alive or require custom-allocators and callers to guess how much storage to reserve for the allocator to pass to child operations. This makes it challenging to compose operations efficiently.
>
> — Weakly Against

> A lot of recent change showing it isn't really baked. GPU use cases in particular have only very recently come up.
>
> — Weakly Against

> The proponents of the Net.TS postulate that it is, while SG1 and GPU experts say that it is not, which ties in with my (somewhat limited) expertise. I am voting weakly agains because while I can not disprove NetTS, I have limited confidence. Furthermore the NetTS's async model does not have genericity as its goal, which I think is crucial for a composable Basis.
>
> — Weakly Against

> Composition of asynchronous operations should be front and center in the design.
>
> — Weakly Against

> Based on what I heard the Networking TS async model works well with networking but there are concerns about other areas.
>
> — Weakly Against

> The async model in [P2444R0] focuses on how work is executed and wants to create an API for that. That has proven very difficult to do over the years because there is such a variety in the capabilities of executors once you move beyond std::thread.
>
> — Weakly Against

> I've used it for a general model, and it's useful, but not anything more than just another library.
>
> — Weakly Against

> Not having worked much with GPUs myself, I'm inclined to defer to vendor's position's (though it would be nice to have a paper explaining that conclusion) in the absence of convincing evidence to the contrary (and I don't find the toy examples given in [P2469R0] to be such evidence). I also find the proliferation of completion tokens (both in number of tokens and number of uses) concerning.
>
> — Weakly Against

> The points brought up in [P2464R0] about composability give me pause about adopting this as our async model.
>
> — Weakly Against

> Though the Asio model has proven history that demonstrates it is *usable* for networking, it has always seemed awkward to me in practice, so I wouldn't say it is a *good* basis for networking. In addition, it is primarily networking-focused, and is not a good fit for other asynchronous code.
>
> — Weakly Against

> I have been convinced that the error handling around work submission is not sufficient as a building block.
>
> — Weakly Against

> I don't think ASIO is a good basis for a *standardized* generic asynchronous computation/communication model. It's quite good at what it does currently, but it's simply too much of a local optimum for these usecases, and doesn't sufficiently meet others.
>
> — Weakly Against

> While there is an existence proof of Networking TS/Asio working for networking, I do not believe it's a good basis for general parallelism, concurrency, or GPUs.
>
> — Strongly Against

> The Asio async model is not a fully developed model that people can use today. It attempts to be the basis of some future machinery people can actually use. Furthermore, it does not allow capturing/representing all asynchronous work as a single task graph necessary for kernel fusion.
>
> — Strongly Against

> We has done its homework here. We knows this not to be true.
>
> — Strongly Against

Asio is a popular and robust library _for_ _networking_. However, Asio supporters show little interest in supporting use cases outside their own narrow domain, unlike [P2300R2] supporters. Also, Asio's flexibility makes it more likely that users will choose easy but wrong ways to manage object lifetimes and concurrency.

On one (I think the first) of three LEWG sessions on the Networking TS and Asio, the Asio proponent began a 90-minute talk on the Asio asynchronous programming model. They used that phrase -- "asynchronous programming model" -- yet had no examples from parallel computing. The apparent duration of their talk showed that they had no interest in learning more. I contrast this with [P2300R2], where the authors' use cases included I/O and networking.

It was startling to see firsthand, what I had heard from a friend in the parallel computing community who is not normally involved in WG21: that Asio supporters have no interest in supporting use cases outside their own. In further mailing list discussion, Asio supporters showed that they did not want to consider use cases from networking and I/O on computers with limited memory or other resources.

I know this sounds like an ad hominem argument, but we're dealing with complicated proposals that are different mainly in the subtle details. We voters need some confidence that a proposal's authors actually care about use cases. Asio proponents' behavior does not give us that confidence.

Regarding Asio's flexibility, remarks by Kirk Shoop and Niall Douglas on the reflector influenced my views. My understanding is that Asio supports many different ways to manage object lifetimes and "do parallelism." As a result, less experienced users reach for the easy thing, shared_ptr. This makes it impossible to reason about object lifetimes. Senders/receivers, by contrast, _only_ supports one way to reason about object lifetimes. This requires more effort, but leads users down the correct path.

On 2021/09/21 at 15:08, Kirk Shoop via Lib-Ext wrote:

> Using shared_ptr is adhoc GC. I too used this to create async lifetimes. I knew it was wrong. Fortunately I met Lewis Baker thanks to Eric Niebler and Lewis explained how to structure lifetimes in async code without adhoc GC. His solution recovers the lifetime management that is core to C++ as a language. This solution is baked into sender/receiver and allows an async operation to pass a raw reference into another async operation, secure in the knowledge that the reference will be valid for the lifetime of the other async operation. This restores local reasoning for lifetimes to async code in C++. The idea that C++ would introduce an async pattern that discards this core principal of C++ is surprising.

On 2021/09/22 at 08:29, Niall Douglas replied:

> > To add to that valid observation, if a programmer is lazy, ASIO's design tends to lead naturally to the use of shared_ptr or some other atomics-based reference counting mechanism. Only by working harder than lazy would one invest the effort to neither shared_ptr nor reference count to manage lifetime in ASIO. Good devs do invest that effort, and here tends to have more good devs than is normal.
>
> Most devs however are lazy, so correspondingly most ASIO code uses shared_ptr. Anyone here on WG21 with enough experience with ASIO will be able to recount stories of having to debug somebody else's ASIO code, and ultimately finding the cause of the (usually random and hard to repeat) failures ultimately came down to the choice to use shared_ptr.
>
> Sender-Receiver requires more typing to use in my experience, so lazy devs will dislike it, but its design tends against the use of shared_ptr. Rather, you tend to set up graphs of Sender-Receiver and you execute some or all of that graph. Shared ptr or reference counting isn't particularly useful in that design model.
>
> — Strongly Against

> I have found very hard to write composed operation with asio, as they need to deal with allocations, the graph of operations being built from the outside-in.
>
> CompletionTokens make the API more cumbersome than it needs to be, each call having to be told how it will be consumed. Error management is somewhat okay in the context of networking but seems very tailored. Cancellation is also cumbersome compared to stop_tokens - despite the recent addition of cancellation slots. (although that is not in the TS)
>
> — Strongly Against

> It isn't. [P2469R0] page 8 inclusive scan is a perfect example. The algorithm implementation of the networking TS does not run efficiently on heterogeneous systems, while the algorithm implementation of [P2300R2] can run efficiently on CPUs, GPUs, or simultaneously on both.
>
> — Strongly Against

> [P2444R0] doesn't seem to propose a good model for parallelism.
>
> — Strongly Against

> I am perfectly willing to believe that the NetTS model is well suited to networking and asynchrous I/O more generally. However, I have seen no evidence that it is well-suited to parallel algorithms, GPUs, and so on. In fact, the examples in [P2469R0] suggest that it probably is not suitable.
>
> — Strongly Against

> I've used Asio for both traditional and HPC network programming in the past. It was fairly adaptable to use cases that it wasn't designed for, although making it support RDMA style communication was tricky. The proactor model is very solid, and I think it's a good model for a lot of network applications.
>
> However, I'm far from convinced that it is universal or general. It doesn't support rich error channels or cancellation. It's also not well suited for structured, compositional parallelism - e.g. building task graphs or dataflow programming. Additionally, it is unclear on which execution contexts certain operations execute, such as error handling. I do not think the Networking TS/Asio model gives users as much control over where things execute as other models, such as senders/receivers.
>
> It's certainly not suitable for GPU programming; it has no notion of bulk execution, and the composition model would require either transfers of control to the host or dynamic parallelism, neither of which are efficient on GPUs. Finally, it does not support lazy execution without additional overhead (allocation or synchronization).
>
> — Strongly Against

> It is my opinion that if we are going to have a "grand unified async model" which the committee has indicated it wants, networking should be built on top of senders and receivers (S&R). S&R are to async execution/parallel algorithms what iterators are to algorithms. It does not seem at all clear to me that the Networking TS/Asio model is capable of what S&R is.
>
> — Strongly Against

> n/a
>
> — Strongly Against

> Firstly, Networking TS does not provide a model for representing asynchronous task, and instead supports variety of them. Without committing specific model, each use cases may follow incompatible APIs (networking, parallelism, GPUs) and not be composable with each other. In other words, the Networking TS, supports variety of different models, but as consequence does not define basis.
>
> Furthermore, the Networking TS/Asio models, does not provide a good support for the error/done/abort handling. Firstly, it does not provide good handling for scheduling error. While for the i/o cases, (that runs mostly on CPU terminating) in case of schedule error may be reasonable choice, this does not apply for more heterogeneous context.
>
> Secondly it merges this channel into existing error_code API, this have major drawbacks:
>
> - Values still need to be provided, even in case of errors - single signature is called in both cases.
> - All task/algorithms needs to be aware and check for special values (abort, error). With separate error/done channels, you not have this problem.
>
> — Strongly Against

> It doesn’t support generic programming, which ultimately restricts what its applications can be. I remain unconvinced that it offers programmers more "freedom" to do whatever, and think it offers less.
>
> — Strongly Against

> ASIOs implementation is really good. ASIOs design was idiomatic and even exemplary when it was started (circa 20yrs ago)
>
> In my opinion ASIOs design is not a good fit for the Standard Library and language in 2021.
>
> If the goal of standardizing ASIO is that all existing code that uses ASIO recompiles cleanly with the std library once NetTS is merged, then the NetTS is the right thing to standardize.
>
> If the goal is to add networking to the standard in C++23 regardless, knowing it is not an idiomatic design and knowing that an idiomatic design for async will be added as well, then NetTS is the right thing to standardize.
>
> If the goal is to add an async pattern to the std, for all async code, that is idiomatic in 2021, then the NetTS does not do that IMO.
>
> If the goal is to add an async pattern to the std for networking that is idiomatic in 2021, then the NetTS needs to update to a design that is currently idiomatic IMO.
>
> — Strongly Against

> As explained in [P2464R0], the ASIO model is not foundational enough. In the process of providing its extremely simplified "executor" abstraction, it loses the information about scheduling errors, which makes layering senders on top of it infeasible.
>
> Additionally, it has not been demonstrated that the ASIO model is capable of expressing arbitrarily complex DAGs of computational nodes in a way that makes it a suitable or even barely acceptable for GPU programming.
>
> — Strongly Against

> The Networking TS model doesn’t facilitate composable error (or payload value) handling. Worse, it’s fundamentally incompatible with such a model.
>
> — Strongly Against

§ 4.2. Poll 2: Sender/Receiver Model Is A Good Basis

> The [P2300R2] model provides a balance between having flexibility and establishing a common API and vocabulary for different use cases.
>
> — Strongly Favor

> Contrary to popular belief this is not a fundamentally new design. It has extensive usage experience in multiple languages and multiple companies and multiple shipping products.
>
> The changes in design from those other implementations (which are largely based on GC lifetimes) are to the names of the concepts and CPOs, compile-time vs. runtime polymorphism, and the lifetime model.
>
> I have extensive experience writing code to this model and it is the only model I have seen that can reduce common errors in async code. That is why I have invested in this, to create an environment where bugs are less likely to write.
>
> — Strongly Favor

> Based on what the API and how I'd use [P2300R2] async model for algorithms it seems to be a good fit. I have implemented basic networking and that seems to work well, too.
>
> — Strongly Favor

> The sender/receiver model is equivalent to continuation passing where continuations can be composed via the contexts/decorators that model the sender concept, and separating the concerns of writing application functions and transforming the functions into continuation passing style. This is a sound and well understood model for computation, including async computation.
>
> — Strongly Favor

> I’ve become confident that the sender/receiver model is the right set of generic abstractions for us to build asynchronous interfaces upon. It has first-class support for rich error channels and cancellation, which are must-have features to certain users. It has very clear semantics about what execution contexts operations occur on. It’s based on lazy execution, but supports eagerization, which is the correct design, as eager can be efficiently implemented on top of lazy, but not vice versa. It also has a notion of bulk parallelism, which is essential for supporting GPUs. We can also extend senders/receivers in the future to support asynchronous streams (e.g. senders that send multiple values).
>
> I view the sender/receiver model as the generalization of the future/promise model that has been pervasive throughout the C++ ecosystem (not `std::future`/`std::promise`, but the general idea). The future/promise model with `.then` continuations is good, but it had limitations - it assumed the existence of a shared state (which implied synchronization and allocation), it didn’t really support cancellation, it was unclear what execution context operations occur on, it doesn’t have a notion of sending multiple asynchronous values, etc. Senders/receivers builds on the proven future/promise model, addressing these faults and giving us something we can truly call a "grand unified model" of asynchrony.
>
> — Strongly Favor

> It is a very good basis however I personally find the flexibility of the model cumbersome. When used in a production code, I will probably have to wrap it it into simpler domain-specific blocks, to hide some of the complexity and bind my choices to the domain.
>
> — Strongly Favor

> I’ve applied it to networking-type problems, I understand its theoretical underpinnings (thanks Steve Downey!) and I trust the combined nvidia folks that it fits GPUs. That does it for me.
>
> — Strongly Favor

> This has been sufficiently demonstrated.
>
> — Strongly Favor

> [P2300R2] is designed with generic programming in mind. Generic programming gives us both composition and the ability to mathematically reason about our programs.
>
> — Strongly Favor

> [P2300R2] seems to covers all cases listed.
>
> — Strongly Favor

> The Sender/Receivers define a model for the representing the asynchronous operations, and I believe that is only thing that actually *needs* standardization, as then third party implementations of concrete use cases (parallel algorithms, networking) can use it, and interoperate without requiring large amounts of glue code.
>
> However, I emphasize the word basis in the poll. This does not mean that it currently have all knobs required for each use cases, but it have mechanism for adding them. The design of `tag_invoke` allows introduction of queries/operations that will work with provided generic algorithms, as all adapters are required to propagate unknown tags.
>
> — Strongly Favor

> I'm convinced that S/R is the direction where we should go, even though the details will need more polishing.
>
> — Strongly Favor

> The sender/receiver model has been designed as a general purpose asynchronous code framework, and I think it achieves that goal in a clean manner.
>
> — Strongly Favor

> The sender/receiver model has been created for the purpose of handling asynchronous events like user interactions or async IO, so it is clear that it is a suitable abstraction for those. Our experimentation shows that it is also particularly closely mapped to certain GPU programming models (to the point that one of our architects, without prior knowledge of senders, said "this looks like CUDA graphs" in the middle of his introduction to the model). We also know what extensions to the bulk interface we want to provide to enable complex parallelism use cases.
>
> — Strongly Favor

> Every constituency we serve should be, at minimum, satisfied with this model. It is difficult to achieve this much, and I believe it has been achieved. We should let C++ be unblocked.
>
> — Strongly Favor

> I think `<algorithm>` is the best success story of the standard.
>
> I am afraid by focusing on specific execution contexts and as very narrow set of use case, whatever will provide will have a short lived relevance.
>
> By being primarily a set of algorithms and focusing on composition [P2300R2] is well suited to be more generally applicable. It provides suitable answer to composition integration with coroutines and existing executors, including GUI, RTOS...
>
> Despite the marginal cost, stop tokens are the interface I want to use for cancellation.
>
> Some work remains to be done (language solution for tag_invoke, more algorithms, async streams), but i believe that design is sound in a variety of use cases.
>
> Building the graph from the bottoms up avoid memory management issues and provides a better interface.
>
> More work is needing (we should be reasonable and target 26, replace `tag_invoke`, gain more experience) but this is the foundation i want to build my castle on.
>
> Does [P2300R2] leave some performance on the table in a narrow set of use cases? Maybe, but somehow I doubt that people for this that is a concern would ever consider using a standard facility to begin with.
>
> Is [P2300R2] a gamble? Maybe a bit, but for one I would like to see the C++ standard not lag decades behind the industry. I have no doubt that [P2300R2] draws many lesson from usage experience with the networking TS, and we should act on this experience, not standardize something because it has been around for a very long time.
>
> — Strongly Favor

> The sender/receiver model is fully formed and usable by developers today. It supports the necessary mechanism to construct/represent an entire task graph that supports parallelism and kernel fusion.
>
> — Strongly Favor

> Sender/Receiver is primarily a set of concepts that we can build a lot of low-level designs on. We know it works for networking-like use cases based on the application Facebook has made of it, and there's nothing obvious that would preclude it working for the socket level.
>
> — Strongly Favor

> [P2469R0] page 8 inclusive scan is a perfect example. The algorithm implementation of [P2300R2] runs efficiently on heterogeneous systems, while the algorithm implementation of the Networking TS does not.
>
> — Strongly Favor

> Of the two models presented in these papers, it is the only one that can address a broad range of needs.
>
> — Weakly Favor

> I'm bothered by the lack of a solution to reentrancy (breaking call stacks in asynchronous loops) and how undifferentiated set_error is as a channel and therefore can't vote strongly in favor.
>
> — Weakly Favor

> I think this is a good basis for paralellism/GPUs but can't judge its suitability for networking.
>
> — Weakly Favor

> My Poll 1 comment is relevant here. Many of the [P2300R2] coauthors and proponents are actively involved in parallelism and GPUs. They also care about supporting use cases outside those domains, including networking and I/O. [P2300R2] has not been around as long as Asio and hasn't been "tried by fire" in the networking domain. I'm pretty familiar with special cases of networking for parallel computing, but not generally familiar with "doing networking in C++," so I'm voting WF instead of SF.
>
> — Weakly Favor

> I am in favor of continued work in this area. Since the areas where async may be useful do not necessarily overlap (parallel may not need networking, networking may not need GPUs), we may just end up with two models.
>
> — Weakly Favor

> The sender/receiver model appears to provide a high degree of consistency and compatibility between asynchronous execution facilities.
>
> — Weakly Favor

> The sender/receiver model focuses on how to define work and how to chain work together, and leaves the gory details of how to execute the work to the scheduler that is chosen. That is an easier API to design and to use.
>
> — Weakly Favor

> This direction is an actual advancement in terms of an async framework, and what it gives you is quite different than asio.
>
> — Weakly Favor

> I find it hard to decide which is superior. I prefer [P2300R2] as it is more comprehensible.
>
> — Weakly Favor

> P2464 make a valid case on why this is a good model for most async tasks, especially with regards to composition.
>
> — Weakly Favor

> While the examples in the Paper are somewhat jarring in their unfamiliarity/complexity I am confident that a good and userfriendly APIs can be build on top of this API.
>
> — Weakly Favor

> [P2300R2] is a nice interface, but it defines nothing of the underlying execution context. If anything, it seems like the P2444 could be a basis and a set of tags used as schedulers and both could co-exist. If the goal is to select one, the interface of 2300 is preferable, but incomplete in some non-trivial ways.
>
> — Weakly Favor

> While it is not yet clear how best to implement loops, especially those with unbounded numbers of iterations, in terms of senders and receivers, the basic tools in [P2300R2] are well matched to the operations needed to express parallel algorithms. While practical network programming will of course require additional primitives as well as additional layers implementing something like an event loop, the direct mapping from synchronous algorithms to sender/receiver structures seems amenable to such additions.
>
> — Weakly Favor

> I think the general direction of sender/receiver more closely matches the semantics of the language and normal functions with regards to its handling of separate value/error channels (return-values and exceptions)
>
> However, I do not think that sender/receiver is sufficiently well-baked to be included in C++23, and could do with some more refinement to address issues raised and more implementation experience.
>
> — Weakly Favor

> My vote is weakly in favor as I am not familiar enough with the networking requirements to be sure that it satisfies those, but for the other domains I am confident it does. If this were only for the parallelism and GPUs use case I would vote strongly in favour.
>
> — Weakly Favor

> There are many more asynchronous use cases than sender/receiver covers, and for some of them S/R is missing important features.
>
> — Neutral

> It shows potential. The complexity is holding it back. I suspect that new language features will be needed in order to get the complexity to a manageable level.
>
> — Neutral

> I lack info on [P2300R2]'s model - specifically of how bulk execution will be preformed and "partial failure" will be handled in the sender / receiver model.
>
> — Neutral

> I have not yet managed to understand the proposal with enough clarity to comment meaningfully.
>
> — Did Not Participate

> Didn’t participate in the discussions, so I don’t have a worthwhile opinion.
>
> — Did Not Participate

> I don’t understand sender/receiver, and haven’t tried to understand them.
>
> — Did Not Participate

> Uninformed and actively abstaining for informational purposes.
>
> — Did Not Participate

> Seems like a very good framework for most use cases, but is higher level p2444, so is less applicable as "a basis"
>
> — Weakly Against

> It can be a basis for that, but I have enough applications with different needs, thus I would oppose standardizing it as the only available model.
>
> — Weakly Against

> Neither Sender/Receiver nor Asio/NetTS are currently a good basis for bulk parallel compute. p2300 does not give me enough information to properly evaluate the rest of this question.
>
> — Weakly Against

> I am concerned about the removal of possibly eager execution without a clear, good path of reintroducing it in the future. We have heard from various stakeholders that eager execution is key to performance for some use cases. I am also concerned about the lack of queries that we would need to generically implement parallelism in a performant way. Finally, there is no up-to-date proposal that describes how [P2300R2] will intercept with the STL algorithms that currently received execution policies. For us, this is a key use case, and without that piece defined we can only speculate about it.
>
> — Weakly Against

The model proposed in [P2300R2] looks good for asynchrony but I tend to disagree that it's good for parallelism (mostly compute one) as well as parallelism on GPU (may be still good for GPU offload and wait for completion, though). Even if the direction looks fine there are missing parts IMO.

First of all, there are no query APIs. User knows nothing about execution context resources, nested parallelism support, etc. to build performant implementation of compute parallelism on top of that. The paper claims that it maps on SYCL programming model but SYCL does have query APIs to get additional information about the device and we use that in our code for the sake of performance.

The second thing is [P2300R2] no longer supports possibly eager execution (except some way to express eagerness with ensure_started) that makes it less friendly for parallelism on CPU and in some cases on GPU as well. I've often heard from users that they are interested in asynchrony but they also want to start the execution for individually submitted parts of the work as soon as possible and wait for completion of all those in one place.

The third thing. If I understand correctly removing eagerness support makes it much harder to build dynamic dependency graph when user may add more work on the fly to the already executing graph. That makes it harder to express "task"-based parallelism.

Finally, we (as the C++ standard committee members) have some ideas how it might interoperate with C++ parallel algorithms but there is no clear definition. I would be very careful with adding something into the standard with the for asynchrony that also targets to support parallelism well unless we have the clear understanding how this programming model interoperates with the rest relevant C++ standard library parts.

One can say "all that listed can be added later" and I tend to believe that technically it's possible but practically I consider the things above essential to support at once in one standard release (C++23 or 26 or whatever) (not necessarily in one proposal) because C++ is quite low-level language that should give reasonably good performance. Lacking the things that helps with performant implementation looks like showstopper to me.

Another counter-argument to what's I've said about performance could be that "scheduler algorithms can be customized" but I think in that case we loose the value of genericity. I anticipate (but do not claim for 100%) that most implementations of algorithms that use schedulers would be customized for known types basically using those schedulers as the tags with ignoring the rest of APIs right to provide efficient implementation. In that case of treating schedulers as the tags we don't even need such complexity that we have in [P2300R2].

— Weakly Against

> The [P2300R2] model makes specific design choices and tradeoffs that are poor, or possibly even unsafe, in the context of networking.
>
> However, a comment on the poll itself: It appears to presume that we must have a single asynchronous model. In the previous LEWG call dated 2020-09-28, a poll was taken on the question "We must have a single async model for the C++ Standard Library". There was no consensus, and that question was not re-asked before taking this poll.
>
> — Strongly Against

> [P2300R2] is opinionated to the extent that it will be rejected by the financial industry and industries involving real-time systems. While it is undoubtedly clever, and currently favoured by the company that the chair works for, it is not a basis for standardisation.
>
> — Strongly Against

> Senders/Receivers is not foundational, it is high level. The DSL it aims to enable is certainly interesting and useful, but as a higher level abstraction it can and should be layered on top of the more foundational Net.TS/Asio. Furthermore, there are MANY operating systems and platforms which expose executor functionality which is identical to what is provided in Net.TS/Asio.
>
> — Strongly Against

§ 4.3. Poll 3: Stop Pursuing The Networking TS

> While I believe the core of Asio model is good, although not suitable as basis for asynchrony in the C++ Standard Library (see poll 1), I do not think we should proceed with the Networking TS design. The design of the Networking TS is very opinionated, and the facility itself is quite large - so large that Library Evolution struggles to build institutional knowledge about it. While an opinionated design is fine for a 3rd party library like Asio, a Standard Library component needs to be universal and consistent with the rest of the Standard Library. I'm not convinced that the Networking TS is.
>
> The Networking TS was designed in a different era (2015-2018); C++ Standard Library design has evolved a lot since then, and I do not think the Networking TS has kept up. In particular I think there are a lot of questionable practices in the Networking TS regarding object lifetimes and value categories. It is likely that there are many modernizations that we'd need to make.
>
> Additionally, the Networking TS uses type erasure and inheritance in a number of places. This is very hazardous in the C++ Standard Library, given our current stance that we must preserve ABI stability. For something like networking, where security is a concern and best practices change over time, we must be able to fix our mistakes and evolve the facility after it ships. I think that requires a design that plans for ABI resiliency from the ground up.
>
> Finally, and most fatally, I think it is completely unacceptable for us to ship a networking facility in Standard C++ that is not secure by default and designed for future evolution to correct security flaws. The world is moving towards TLS as a requirement - many websites can only be accessed through https for example. If we standardize a networking facility that is not only insecure by default, but has no support for secure communication, it will not age well. 10 years from now, we may find that facilities that do not support secure networking are entirely unusable. There's a reason the industry is moving to secure by default - security flaws can have major impacts on human lives, property, and the environment. Insecure systems can literally get people killed.
>
> — Strongly Favor

> The Networking TS is not an universal abstraction for both parallelism and concurrency. Therefore its pursue requires introducing multiple async models into C++.
>
> — Strongly Favor

> We should stop pursuing the Asio model, get S&R in C++23 or 26 and then after that is done, start working on a Networking library that is built on top of S&R.
>
> — Strongly Favor

> Pursuing both [P2300R2] and P2444 seems like a waste of time. Given my answers to the other polls, I think this answer should be unsurprising.
>
> — Strongly Favor

> We need to learn to crawl before we can run. Why we try to shove a network library in the standard when we don't have the basis figured out is beyond me. Why sockets when we can barely open a files and can't deal with processes? [P2300R2] would cater to all use cases.
>
> The Network TS represents a lot of work by dedicated people, and I'm sure we will be able to adapt the socket/ip machinery, with modifications, to [P2300R2].
>
> But, right now, we need to focus on progressing [P2300R2]. We don't have the bandwidth to pursue multiple directions.
>
> More importantly, we shouldn't hate implementers and users so much as to force on them the burden of two different non-composable models. This would only contribute to make asynchrony in C++ worse, not better. We need to think in terms of ecosystem here, not in term of shipping disjointed features so we can cross them out of our Christmas list.
>
> Beside, there is no pressure for us to ship a small subset of a widely available library.
>
> — Strongly Favor

> The Asio model is redundant with the sender/receiver model and the sender/receiver model is more fully baked and amenable to parallelism.
>
> — Strongly Favor

> The ASIO design is dated and not desirable for inclusion in Standard C++ in 2021. Additionally, as P2464 notes, the model is not as foundational as senders.
>
> — Strongly Favor

> It is not useful to pursue a design that doesn't fit into a bigger picture of a general asynchrony model.
>
> — Strongly Favor

> ASIOs implementation is really good. ASIOs design was idiomatic and even exemplary when it was started (circa 20 years ago).
>
> IMO ASIOs design is not a good fit for the std library and language in 2021.
>
> If the goal of standardizing ASIO is that all existing code that uses ASIO recompiles cleanly with the std library once NetTS is merged, then the NetTS is the right thing to standardize.
>
> If the goal is to add networking to the standard in C++23 regardless, knowing it is not an idiomatic design and knowing that an idiomatic design for async will be added as well, then NetTS is the right thing to standardize.
>
> If the goal is to add an async pattern to the std, for all async code, that is idiomatic in 2021, then the NetTS does not do that IMO.
>
> If the goal is to add an async pattern to the std for networking that is idiomatic in 2021, then the NetTS needs to update to a design that is currently idiomatic IMO.
>
> — Strongly Favor

> The security point is the major sticking point. I would be willing to accept a networking-specific async model.
>
> In 2021 it would be irresponsible to ship networking facilities without an appropriate security mechanism (TLS in this case). However, I do not believe WG21 can react appropriately to security problems with the constraints we have in place. This combination means that I will be strongly against putting almost any networking proposal in the standard, regardless of the API or async model.
>
> — Strongly Favor

> The networking TS does not adequately cover what many programmers refer to as networking, so it's not even a solution the thing its name suggests it addresses.
>
> — Strongly Favor

> The approach proposed in [P1861R1] is superior in many ways which are outlined in that paper.
>
> — Strongly Favor

> I think it's better to concentrate our efforts in one direction rather than pursuing multiple options when one seems more favorable.
>
> — Weakly Favor

> I believe a future evolution may work out but there are several design elements that need revisiting.
>
> — Weakly Favor

> We need networking, but I am wary of standardizing something based on the Asio executor model, given that people will want to use the same asynchronous execution model for all code.
>
> — Weakly Favor

> I was particularly influenced by some discussion on the mailing list:
>
> > I still don’t understand why we insist on conflating i/o with asynchronicity or concurrency. We should start with multiplexing i/o on a single thread with a submission and completions queue: you add items to the submission queue if there is space (otherwise you defer to later), you reap completions from the completions queue. That’s all we need for Networking as a bare minimum viable. And that model works beautifully in a 2Kb RAM CPU, never mind a 128Kb RAM CPU.
> >
> > You might note that io_uring and Windows RIO literally do exactly what I just described. So did POSIX aio, though poorly.
> >
> > Sender-Receiver naturally fits a single threaded submission-completions queue model. And not a single mention of asynchronous nor concurrent anything is needed here.
>
> This reminds me a lot of a second-hand story about an MPI (Message Passing Interface for distributed-memory parallel computing, a 30-year-old industry standard that has its issues but works just about everywhere) expert who could not manage to convince the Asio folks to care about his use case. The second sentence in Niall’s quote above is not very different from how MPI messages work. It does seem like there’s always some assumption in Asio folks' minds about having extra compute resources to make progress on networking tasks in the background. This does not align with C++'s goal to work on all sorts of hardware.
>
> — Weakly Favor

> I don’t think there should be two async models in the standard. Given my votes on 1 and 2, this one follows.
>
> — Weakly Favor

> It’s time to consider simpler, fresher approaches.
>
> — Weakly Favor

> Stop pursuing but with the caveat that the gap between this and Sender/Receiver probably isn’t vast. That gap might be closed without throwing away the low-level parts of the the design, though that needs work.
>
> — Weakly Favor

> The answer is for the very specific to the design of the Networking TS. I believe that ASIO based model refactored into providing native support for sender/receivers/scheduler.
>
> — Weakly Favor

> While I would have little qualm in referring someone to Asio or the NetTS flavor of the library, as it clearly works, I do not believe that it provides a sound composable vocabulary for the standard library. I do not have confidence that I could compose two independent "third-party" libraries using these tools, or third party components with my own code.
>
> I certainly do not want to spend any more time seeing if the Network TS design can be refactored or extended to support other use cases, and any further work should be solely focused on wording that reflects the design as is.
>
> I am not concerned about singularity or purity of the standard library. We have several different models of IO and of text formatting.
>
> — Weakly Favor

> I would like to see a networking design that uses the same async model used for other parts of the standard library and I’m not convinced that NetTS is the right async model for the standard library.
>
> — Weakly Favor

> (parts of) asio are quite suitable for networking. But I think asio is better off as its own library.
>
> — Neutral

> There is plenty of history that users could pull from if this is in the C++ Standard Library. For users that don’t need asynchronous operations, the design seems acceptable. As someone maintaining an implementation of the sockets portions of the TS without async (not needed with our scheduler/didn’t want to implement twice) and against a custom network stack, there are no concerns about implementability.
>
> Since C++23 was removed from the poll, this could easily be weakly against stopping work on the Networking TS.
>
> — Neutral

> The implementations of the specific contexts like sockets are good, and they have a lot of experience mapping hardware to reasonable primitives. The primitives are just too general to be composable, but they *are* good, so should be used to implement sender/receiver/scheduler interfaces. We should stop using the composition mechanisms though.
>
> — Neutral

> Since Asio is an established design targeting the area of networking with extensive implementation experience, there isn't much "work" to do for it. As such, it's not particularly valuable for LEWG to explicitly disrecommend such work; whether or not the Networking TS is merged into the IS, authors may continue to propose refinements of the sender/receiver model, and committee members may continue to evaluate its suitability for networking applications. Nonetheless, consensus here might usefully encourage such refinements.
>
> — Neutral

> In its current form. Yes. I do not think that the TS is fundamentally wrong though, and I think it should not be to hard to "rebase" it onto senders/receivers.
>
> — Neutral

> I can imagine the NetTS design could still be a useful basis for standard C++ networking. I can also imagine that other better candidates could be proposed. It remains the best candidate we have at this moment.
>
> — Neutral

> The Networking TS has significant field usage. However, I believe it should be composable with whatever asynchronous model is settled on for C++. If it is shown to either be composable with that model or integrates that model directly in its design, I would be more likely to support it. However, without this settled, I'm neutral.
>
> — Neutral

> I am not expert in Networking domain and I think there are people that better know what is good answer for networking and what is not. As I wrote in the comment for the Poll 1 the whole programming model might be interesting with some tweaks if we consider this question broader that just "the C++ answer for Networking" (and thus, I want to vote rather than abstain). But with the current wording of the poll I don't feel comfortable to have "For" or "Against" opinion.
>
> — Neutral

> I don't know the Networking TS specifically or networking in general well enough to have an informed opinion about whether or not the Networking TS it is a good design for standardized networking.
>
> — Did Not Participate

> I don't feel familiar enough with networking requirements and the Networking TS to have an opinion on this.
>
> — Did Not Participate

> I haven't been present for sufficient discussion to have context on this. It would be good to get something into use sooner rather than later, but I don't know the ramifications of trying to do that.
>
> — Did Not Participate

> Didn't participate in the discussions, so I don't have a worthwhile opinion.
>
> — Did Not Participate

> I am still uncertain of the competing arguments which, although presented, I struggle to understand.
>
> — Did Not Participate

> Uninformed and actively abstaining for informational purposes.
>
> — Did Not Participate

> As mentioned, seems to me "Asio" is the right direction, but I would love to see more usage experience (therefore I will prefer to encourage more work, or at least, more information collection). Personally, I’m OK with delaying the progress of standardizing this till such info is collected, but I may be missing a pain exists in using non-standardized alternatives for few more years.
>
> — Weakly Against

> I want networking in the standard. I don’t think having support for both models is inherently bad.
>
> — Weakly Against

> I think the authors likely know what they are doing and this has been so long in the works.
>
> — Weakly Against

> Other than programming sockets directly, nothing else has the field experience of ASIO. Before ASIO, WG21 tried coming up with its own networking API from scratch, but didn’t succeed.
>
> — Weakly Against

> There are facilities/capabilities within the ASIO design’s that are not available via the sender/receiver model, so abandoning it would run the risk of forcing C++ Standard Library users to go elsewhere in order to obtain flexibility/performance.
>
> — Weakly Against

> We’ve not seen a better design, nor anything with a route to becoming one, at the present time.
>
> — Strongly Against

> This would be insanity. The Asio/networking model reflects current practice in every async IO library shipped by OS vendors and used by every industry, except the few niche examples mentioned during committee discussions.
>
> — Strongly Against

> For networking in C++ it’s the defacto standard, so it should be properly standardized.
>
> — Strongly Against

> We should standardize existing practice, the Networking TS does that, and, in contrast to the other proposals in play in this series of polls, provides a sufficient basis for me, as a standard library implementer to proceed with an implementation.
>
> — Strongly Against

> Both models follow very different design principles and both are very useful - we should pursue both and look to smooth the differences.
>
> — Strongly Against

> I have not seen any specific proposal for a networking library put forth that may even be considered as a replacement.
>
> — Strongly Against

> The networking TS seems to me a good example of standardising existing practice.
>
> — Strongly Against

> Its been 6 years since Net.TS was published and 18 years of Asio. Inventing something new and pushing it through ahead of the Net.TS to replace it, is quite frankly disrespectful. Not only does it disrespect the author of the TS who is more of a subject matter expert than anyone else in the committee, but it disrespects everyone who wrote papers on top of it expecting a high likelihood of acceptance. The message it sends to the C++ community is terrible: don’t bother participating in WG21 or taking time to write papers, as your work can be discarded on a whim based on shifting political winds.
>
> — Strongly Against

> Asio is 20 years old and battle tested and the networking TS has been in development for years. Scrapping that for innovation would go against the principles of C++ evolution.
>
> — Strongly Against

> An artifact that distills proven deployment experience across such a large swathe of time should not be simply discarded.
>
> — Strongly Against

§ 4.4. Poll 4: Sender/Receiver Networking

> Any further work on asynchronous tasks should fit inside a theory of composition of asynchronous tasks.
>
> — Strongly Favor

> We need to make every level of the design composable, and use the same vocabulary everywhere.
>
> — Strongly Favor

> A consistent set of concepts is useful for composition. This should be the starting assumption, although precisely what it means for the low-level parts of networking is still open. It does mean we should have a well-specified callback interface and a lazy, structured state maintenance strategy.
>
> — Strongly Favor

> If we accept that the sender/receiver model is the better asynchronous execution model then we should be using it as the basis for the asynchronous execution in the networking components of the standard library.
>
> — Strongly Favor

> Just like the STL uses iterators to abstract containers' internals away when computing an algorithm, I’m pretty convinced at this point, that senders and receivers are to be used to abstract the details of where an algorithm is computed.
>
> — Strongly Favor

> I read that it should have native (efficient) support for this model. For me it does not preclude design that supports both sender receivers, and other models if one like that would be presented.
>
> — Strongly Favor

> There should be one model.
>
> — Strongly Favor

> Sender/receivers are suited to both concurrency and parallelism, and avoids having to introduce multiple async models into C++.
>
> — Strongly Favor

> Independently of [P2300R2] we need to figure out:
>
> - What kind of io context we want, probably based on general-purpose system thread pools
> - What kind of features we want to support (timer, files, process, sockets, streams)
> - what kind of network we want (sockets, taps)
> - What kind of security we want
>
> When and if we have answers to these questions, [P2300R2] can provide the API for these features.
>
> I'm not endorsing networking in the standard here. But if there should bet networking, it should be based on [P2300R2].
>
> — Strongly Favor

> The sender/receiver model is generic enough to support asynchronous networking programming.
>
> — Strongly Favor

> Having a common and currently idiomatic pattern to share across all libraries within the std and in the wild is compelling
>
> Having Networking, in the standard, as the odd man out is not compelling.
>
> ASIO & the NetTS are based on types/traits/partial-specialization/private-virtual-interfaces.
>
> Because it is not based on concepts it is impossible to reuse the std::socket type with your own io_context. due to the type-based and private-interface between the distributions io_context and the distributions std::socket type, there is no portable way for a user to replace the io_context or the socket type. The only way to get a new io_context is to convince the distribution to add it and maintain it and ship it.
>
> Specific case. The hot new api for networking (and file, pipe, pretty much any syscall) is IO_URING. A shipped NetTS of the current design would not allow users to portably inject IO_URING support. All users would have to wait for the distribution to write a new io_context and release an update that contains the new io_context.
>
> A NetTS based on concepts for io_context, socket, etc.., where the io_context supplies its own implementations of socket etc.., is easily recompiled against a different implementation.
>
> — Strongly Favor

> I don’t think there should be two async models in the standard. Given my votes on 1 and 2, this one follows.
>
> — Strongly Favor

> The C++ Standard Library should have one unified async model, and it should be based on [P2300R2].
>
> — Strongly Favor

> I strongly believe that the sender/receiver model is the only option on the table that is suitable as a "grand unified model". Personally, I'd be willing to have more specialized models (like the Networking TS/Asio model) for specific components, as long as they could interoperate with the sender/receive model. But, Library Evolution has signaled that we do not want to live in a world where we have multiple asynchrony models in the C++ Standard Library. Therefore, I think the only path forward is one in which Standard Library networking is based on the sender/receiver model.
>
> — Strongly Favor

> The asynchronous facilities in the standard library should be based on a universal abstraction for asynchrony. Sender/receiver is it.
>
> — Strongly Favor

> All of the arguments have convinced me that networking _could_ be based on the sender/receiver model, and that this would be a good approach, even on computers with limited memory or other resources. I'm also convinced that networking could integrate naturally with parallel computing using senders/receivers, and that it's thus worth having a single programming model instead of two. Of the two models being discussed here, I think [P2300R2] is a better choice for this integration use case.
>
> I'm not a "networking in C++" expert, so I voted WF instead of SF.
>
> — Weakly Favor

> A generic composable basis for async operations is fundamentally important for a reasonable programming model.
>
> — Weakly Favor

> I think we should be pursuing async, not networking.
>
> — Weakly Favor

> This seems doable and relatively simple. Also, the unified model will benefit in how users will perceive C++ standard.
>
> — Weakly Favor

> Only weakly in favor because if the standard library does not provide S/R networking, I am confident I could write networking that interoperated with the standard library algorithms, senders, receivers, and schedulers. I would prefer, of course, not to have to.
>
> — Weakly Favor

> I don’thave a full networking inplementation and examples of all relevant uses cases using [P2300R2]. Also, I don’t have benchmarks. I think [P2300R2] can and should be the basis but I don’t have a strong opinion, yet, that it is the best approach.
>
> — Weakly Favor

> Ideally this would be nice, especially if it can be made more clear that the sender/receiver and awaitable models are both isomorphic and intercompatible such that the coroutine-based examples we’re used to seeing would mostly keep working.
>
> — Weakly Favor

> I base this choice merely on an understanding of [P2300R2] rather than a detailed understanding of the surrounding matters
>
> — Weakly Favor

> Maybe it should, but there is no concrete proposal to evaluate, let alone one that has been field-tested. We’ve tried inventing networking before pursuing ASIO, and that effort didn’t succeed.
>
> — Neutral

> Networking should be based on the async model that is chosen as the standard library’s general async model. If that’s sender/receiver then it should be based on that. I would want to see more implementation experience with a sender/receiver-based networking design before being in favour of this.
>
> — Neutral

> It seems that at least a proof-of-principle implementation (maybe even in the form of a paper rather than code) is needed before changing the status quo and establishing a precedent for further discussions. SG4 should surely also be involved in such a decision.
>
> — Neutral

> Given that the NetTS model can interoperate with senders, and that further work could presumably iron out the current wrinkles at the interface between them, I don’t see a compelling case to mandate that networking be based on senders. It is important that they interoperate; it is less important that they be identical.
>
> — Neutral

> I don’t see anything that would prevent this and could easily be in favor. As a developer, I would want a single model. However, it just may not be possible to have one solution for all cases.
>
> — Neutral

> The benefits of the high-level API consistency provided by the sender/receiver proposal are clear, but I have remaining concerns about its flexibility. The degree of disagreement among WG21’s concurrency and parallelization experts means that I do not yet feel comfortable to agree that it "should be" Standard Library concurrency solution. I would like to hear about implementation *and deployment* experience of Networking TS-equivalent library before expressing an opinion on this.
>
> — Neutral

> I think [P2300R2] should be constructed with networking in mind, and I think that it could service that need. I would be strongly against adding the networking parts to the standard though.
>
> — Neutral

> I believe that networking should be composable with whatever asynchronous model is settled on for C++. However, it is not yet decided that [P2300R2] is that model, and so I’m neutral.
>
> — Neutral

> Networking in C++ standard library should be redesigned from scratch so it’s premature to ask this question.
>
> — Neutral

> Assuming the open questions with either model are resolved I don’t see either of them as better or worse than the other and therefore have no preference.
>
> — Neutral

> I don’t have enough experience with networking code to know whether sender/receiver is a good fit for networking. I believe it is possible to use sender/receiver for networking, but I don’t know if it is the best way.
>
> — Did Not Participate

> I don’t feel familiar enough with networking requirements and the Networking TS to have an opinion on this.
>
> — Did Not Participate

> This sounds like another formulation of Poll 3 but less broader. I would like to abstain here and let Networking TS guys to decide what the want to base Networking TS on
>
> — Did Not Participate

> If sender/receiver is compatible with P1861, then I agree. But here too, I don’t understand sender/receiver.
>
> — Did Not Participate

> I don’t feel that I have enough information to determine this is feasible.
>
> — Did Not Participate

> Didn’t participate in the discussions, so I don’t have a worthwhile opinion.
>
> — Did Not Participate

> Ideally we should aim to design basic network operations such that they work with both models. I do realize this might not be possible, although perhaps concepts can help here a little.
>
> — Did Not Participate

> Uninformed and actively abstaining for informational purposes.
>
> — Did Not Participate

> Sender/receiver has too many shortcomings for this.
>
> — Weakly Against

> It doesn't look like there is a fundamental incompatibility between the executor and the sender/receiver/scheduler based approach to me. And there is a real cost for us not getting this stuff moved forward in that we have to deal with hardware vendor specific models and other extensions of C++ all the time. So I rather have Networking getting in and a different (possible slightly redundant/isomorphic) solution for parallelism/GPUs which better expresses things how we want to do stuff there.
>
> — Weakly Against

> I am concerned at what the consequences might be of trying to base networking on a new model at this late stage.
>
> — Weakly Against

> There's no reference implementation or conclusive paper on networking with [P2300R2]. This is opposed to two decades of experience with Asio.
>
> — Strongly Against

> The sender/receiver model is a higher level abstraction than the executor model. Asio has demonstrated that the one can be implemented in terms of the other.
>
> — Strongly Against

> This poll question is a significant design departure from the status quo. No specific proposal has been presented showing how a networking API based on sender/receiver would look, nor what concrete design choices would be recommended, nor why it is required and the status quo is insufficient. At best, we can only infer from [P2300R2], and based on the design choices there I find it a poor, and even unsafe, fit for the domain, particularly when compared to the status quo.
>
> — Strongly Against

> A paper outlining, in detail, what a networking model using p2300 looks like would be necessary to even begin a serious evaluation and discussion of this question. That, to my knowledge does not exist, so as a standard library implementer, I have no basis on which to evaluate the merits of this notion.
>
> — Strongly Against

> Asio/Net.TS provides a more foundational execution model which has not only proven itself in Boost and in the standalone asio but also in different incarnations of the same executor concept on different operating system APIs and frameworks. It is a proven design.
>
> — Strongly Against

> To the best of my understanding, this would unnecessarily pessimize low-latency use cases. Having a completion token that makes the networking TS use senders/receivers, perhaps as the default, seems very valuable. However, making the overhead mandatory and excluding other concurrency models seems undesirable.
>
> — Strongly Against

§ 4.5. Poll 5: Secure Sockets

> Asio demonstrates very neatly that implemented TLS over async sockets is trivial.
>
> — Strongly Favor

> While "secure sockets" are necessary for many modern web applications but they are not necessary for socket I/O to be useful and the latter is a precondition for the former so it doesn’t make sense to hold the latter up for the former (the former can always be added later).
>
> — Strongly Favor

> Network security is obviously of great importance, and "secure by default" is a good design principle. However, it is not useful to delay standardized networking to incorporate specific security technologies. Standard certificate-based security involves a great deal of configuration and dynamic, persistent data that is not easily integrated into or deployed with the standard library. Other technologies (e.g., SSH) would have to be implemented as a layer on top of the standard library regardless, and it does not seem particularly harmful to standardize (only) ordinary sockets that can be wrapped into any of several such layers.
>
> — Strongly Favor

> Not all networking is over the public internet or is physically accessible to the public at large, and many domains don’t have security concerns beyond physically securing their network connections.
>
> — Strongly Favor

> TLS can be build on top of the insecure sockets, and there are applications that do not need encryption. While I am sympathetic to the goal of building "key-ready" solutions, a layered approach is more reasonable
>
> — Strongly Favor

> While it is desirable to provide support for TLS, there are sufficient use cases for socket-based networking without TLS to justify its use and inclusion in the library. Furthermore, finalising the core of the networking library enables the development and specification of TLS support on top of a fixed point, rather than a constantly moving target.
>
> — Strongly Favor

> Asio already puts that as a layer on top of the networking functionality, so we can further explore Standard solutions while third party libraries can provide these layers.
>
> Furthermore, many applications use proxy layers to provide TLS (e.g. nginx) and so they can use the net.ts in a secure way without any need for TLS in C++.
>
> — Strongly Favor

> In low-latency distributed computing the lack of encryption is a very useful feature, since it aids to reduce the latency of network communication. It is a system designer responsibility to provide alternative means of establishing trust between machines.
>
> — Strongly Favor

> The C++ Community needs to accept that using third party libraries is going to be a fact of life. WG21 must reject the excuse that "my company can’t use anything that is not in the standard" as the sole or primary justification for standardizing a library-only component, as it is unsustainable.
>
> — Strongly Favor

> I do not want to ship something that cannot be easily *extended* to support secure sockets, but I don’t think we need to require it now; we are better off shipping the basic framework if requiring secure sockets would hold up shipping networking.
>
> — Weakly Favor

> Build foundations first.
>
> — Weakly Favor

> Assuming that I understand correctly and this can be added later, I think shipping in multiple stages makes sense.
>
> — Weakly Favor

> We discussed this in Belfast and agreed that the feature should be pursued but that sockets could be added to C++ without the feature. Having secure sockets is important for some use cases. Having just a sockets and the vocabulary types that goes with it would give education and closed networks an out of the box solution. We have seen from the gaming community that portions of the C++ Standard Library are not desirable for all.
>
> — Weakly Favor

> I think its ok. If we have a basis to work on extensions we can add secure sockets in 26.
>
> — Weakly Favor

> I am happy with a layered approach, such as that provided by Boost.Asio + Boost.Beast, to support secure sockets using a layering approach. Additionally, many of the uses of networking I have come across in my own experience have not required secure sockets as the networks involved are isolated from general access.
>
> — Weakly Favor

> That can be added afterwards.
>
> — Weakly Favor

> Given how much of the Internet traffic is encrypted these days, the C++ standard still won’t know most of the internet without TLS. But we may not have much of a choice if there are unavoidable ABI problems. It’s not much of a standard library if vendors won’t ship it.
>
> — Weakly Favor

> I don’t necessarily agree with p1860’s premise that networking MUST be secure by default, I do agree that networking should ideally support TLS/DTLS, whether that rises to the level of requirement for initial release is debatable.
>
> — Weakly Favor

> It’s plausible that we could add TLS later, and having standardized sockets would be a good starting point compared to having nothing at all.
>
> — Weakly Favor

> We should try to support SSL but it’s not my usecase, I need multicast UDP.
>
> — Weakly Favor

> TLS has not been API stable enough in the past to require it in the standard library. I do not want to be in an API or ABI argument with my compiler vendor to fix a security problem. S/R gives enough separation of concerns that independent libraries can be integrated into the model without vendors having to do that in core. Asio also has enough separation to provide external support for TLS, and boost.ssl shows that the SSL management interfaces can be externalized.
>
> — Weakly Favor

> C++ has such a long history of having terrible defaults that it seems churlish to get in the way of adding another one.
>
> — Neutral

> I don’t have an opinion on that. To some extent it feels that asking for SSL/TLS support is just stalling the Networking TS from proceeding. Security is important, though. For my use cases it isn’t needed: all I’m working with is either within a secured environemt or transported via a VPN. Further I can’t assess the added complexity for users if security is supported.
>
> — Neutral

> The networking without encryption is still usable in certain situations, however we are shipping the networking as part of standard library to make it easier to use. I understand the concern that providing only unsecure sockets to libraries, will cause increase used of unencrypted connections, in context where they are not recommended/harmful.
>
> — Neutral

> Networking in C++ standard library should be redesigned from scratch so it’s premature to ask this question.
>
> — Did Not Participate

> I do not have an opinion here.
>
> — Did Not Participate

> To me secure sockets are essential because without those we cannot support secure connections and whatnot (if I am not mistaken) that makes (potentially standardized) Networking C++ API less attractive to the user. But I want to rely on the Networking experts decision here.
>
> — Did Not Participate

> I have not been able to keep up with this. The idea that C++ would have to directly support TLS to ship networking strikes me as ludicrous, but someone must have a reason to believe the basis operation has to include this.
>
> — Did Not Participate

> I don’t feel familiar enough with networking requirements and the Networking TS to have an opinion on this.
>
> — Did Not Participate

> Uninformed and actively abstaining for informational purposes.
>
> — Did Not Participate

> I don’t know enough about what a design supporting secure sockets would look like to know whether the current design could be straightforwardly extended to support secure sockets.
>
> — Did Not Participate

> I believe it is outdated and borderline irresponsible to ship a networking implementation (whatever it is) that doesn’t make it easy to get secure networking out of the box. It’s just the way the whole world works now. We should expect that programmers will reach out for the tools we give them in the Standard Library. If they do, and if those tools do not allow them to have secure networking, they will not have secure networking. Is that really something the committee wants to get behind?
>
> — Weakly Against

> I would prefer to see sockets shipped secure by default, with insecurity being a deliberate choice, perhaps via naming e.g. socket and insecure_socket.
>
> — Weakly Against

> Secure sockets are clearly important and I found myself weakly persuaded by the arguments that they should be dealt with as part of the core proposal.
>
> — Weakly Against

> The solutions proposed to add secure sockets to the NetTS later are clunky. These proposed solutions make it obvious that security is an aftermarket addon that was not incorporated into the design at the beginning.
>
> — Weakly Against

> I do think it’s important to have some support for secure sockets out of the box to avoid programmers defaulting to writing insecure networking applications just because that’s what’s available in the standard. However, we have yet to see a compelling proposal for this and I don’t want to wait forever to get some form of networking into the standard library.
>
> — Weakly Against

> C++ gets enough flak as it is for security vulnerabilities.
>
> — Strongly Against

> It is completely unacceptable to ship insecure networking, as explained in P1860.
>
> — Strongly Against

> The uses cases for non-secure networking are too limited to burn our scare resources on that. Beside, we have a responsibility to not point the foot bazooka in the direction of our users.
>
> — Strongly Against

> It is acceptable to ship cars in the United States that do not have seatbelts.
>
> — Strongly Against

> Not in 2021, no.
>
> — Strongly Against

> C++ socker-based networking should be secure by default.
>
> — Strongly Against

> In 2021 it would be irresponsible to ship networking facilities without an appropriate security mechanism (TLS in this case). However, I do not believe WG21 can react appropriately to security problems with the constraints we have in place. This combination means that I will be strongly against putting almost any networking proposal in the standard, regardless of the API or async model.
>
> — Strongly Against

> Of course not.
>
> — Strongly Against

> Not in 2023.
>
> — Strongly Against

> As one of the presenters said at the telecon prior to this ballot, we must think about programs that get exposed to the internet. Almost nobody should be making unencrypted connections over the internet, and shipping a first revision of Standard C++ Networking without encrypted sockets would be irresponsible.
>
> — Strongly Against

> Support for secure sockets should simply be mandatory. There is no acceptable alternative in the modern era.
>
> — Strongly Against

> C++ standard is lagging behind real world. We need to deploy a solution that world with current standards, not with 10-years-old standards. TLS is a must.
>
> — Strongly Against

> I think it is completely unacceptable for us to ship a networking facility in Standard C++ that is not secure by default and designed for future evolution to correct security flaws. The world is moving towards TLS as a requirement - many websites can only be accessed through https for example. If we standardize a networking facility that is not only insecure by default, but has no support for secure communication, it will not age well. 10 years from now, we may find that facilities that do not support secure networking are entirely unusable. There's a reason the industry is moving to secure by default - security flaws can have major impacts on human lives, property, and the environment. Insecure systems can literally get people killed.
>
> — Strongly Against

§ References

§ Informative References

[P1861R1]

JF Bastien, Alex Christensen, Scott Herscher. *Secure Networking in C++*. 11 May 2020. URL: https://wg21.link/p1861r1

[P2300R2]

Michał Dominiak, Lewis Baker, Lee Howes, Kirk Shoop, Michael Garland, Eric Niebler, Bryce Adelstein Lelbach. *std::execution*. 4 October 2021. URL: https://wg21.link/p2300r2

[P2444R0]

Christopher Kohlhoff. *The Asio asynchronous model*. 15 September 2021. URL: https://wg21.link/p2444r0

[P2452R0]

Bryce Adelstein Lelbach. *2021 October Library Evolution and Concurrency Polls on Networking and Executors*. 2022-02-15. URL: https://wg21.link/P2452R0

[P2464R0]

Ville Voutilainen. *Ruminations on networking and executors*. 29 September 2021. URL: https://wg21.link/p2464r0

[P2469R0]

Christopher Kohlhoff, Jamie Allsop, Vinnie Falco, Richard Hodges, Klemens Morgenstern. *Response to P2464: The Networking TS is baked, P2300 Sender/Receiver is not.*. 4 October 2021. URL: https://wg21.link/p2469r0